DSA WEEK 4 - ARRAYS

1D Arrays, Searching, In-Place Manipulations & Complexity Intuition

From how the JVM lays out an array in memory to writing bug-free binary search

Introduction

An array is the first data structure every programmer meets, and the one every other structure is built on. Lists, stacks, queues, heaps and hash tables all keep their data in arrays underneath. If you understand exactly how a Java array sits in memory, you will understand why indexing is instant, why scanning is cache-friendly, and why some patterns (binary search, two pointers) are so powerful.

This week connects three layers: **how arrays work inside the JVM**, **the core patterns** (traverse, find max/min, reverse, pair up, search) and **complexity intuition** so you can look at a problem's constraints and know which approach will finish in time. It builds on last week's setup: you only need a working JDK and the `javac` / `java` commands.

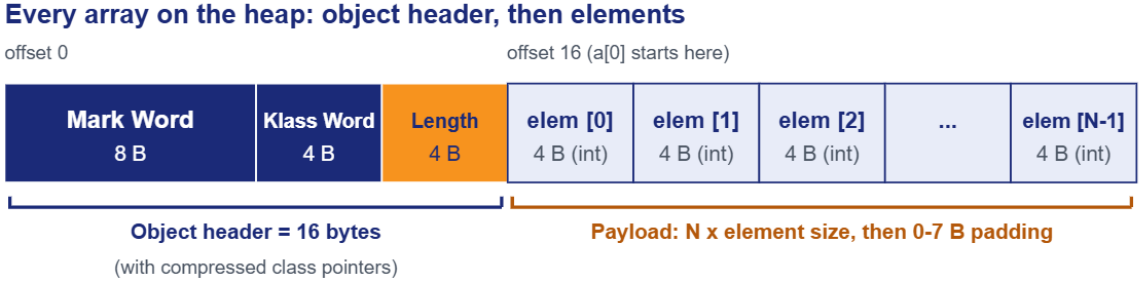
WHAT YOU WILL LEARN

- Describe the heap layout of a Java array and compute the address of any element.
- Explain why primitive arrays are cache-friendly and reference arrays are not.
- Declare, allocate, traverse and read large inputs with a fast scanner.
- Write robust max/min, in-place reverse and all-pairs loops.
- Implement linear search, binary search, lower bound and upper bound without off-by-one bugs.
- Pick a target time complexity from the constraints of a problem.

1. Deep Memory Architecture of Java Arrays

In Java an array is a dynamically created, **first-class object** that lives on the JVM heap. In C/C++ an array name is little more than an alias for a raw address. A Java array instead carries metadata next to its elements, which is what gives Java strict memory safety and automatic bounds checking.

1.1 JVM Heap Layout & Object Headers



Part	Size	What it holds
Mark Word	64 bits	Runtime metadata: identity hashcode, lock state and GC age bits.
Klass Word	32 or 64 bits	Pointer to the class metadata in Metaspace (e.g. [I for int[]). With -XX:+UseCompressedClassPointers it takes 32 bits (4 bytes).
Array Length	32 bits	Capacity stored as a signed 32-bit int, so the theoretical maximum is $2^{31} - 1$. Real JVMs cap it slightly lower (typically $2^{31} - 8$).
Data Elements	N x size	The contiguous payload.
Padding	0–7 bytes	Objects are 8-byte aligned; zero bytes are appended if header + data is not a multiple of 8.

NOTE

With compressed class pointers the header is 8 + 4 + 4 = **16 bytes**, which is the number used in every address calculation below. Without them the header grows.

1.2 Memory Address Translation & O(1) Random Access

The property that makes arrays fast is O(1) random access. Because elements are contiguous and equally sized, the CPU finds index *i* with one multiply and one add:

$$\text{Address}(A[i]) = \text{Base Address} + \text{Header Size} + (i \times \text{Element Size})$$

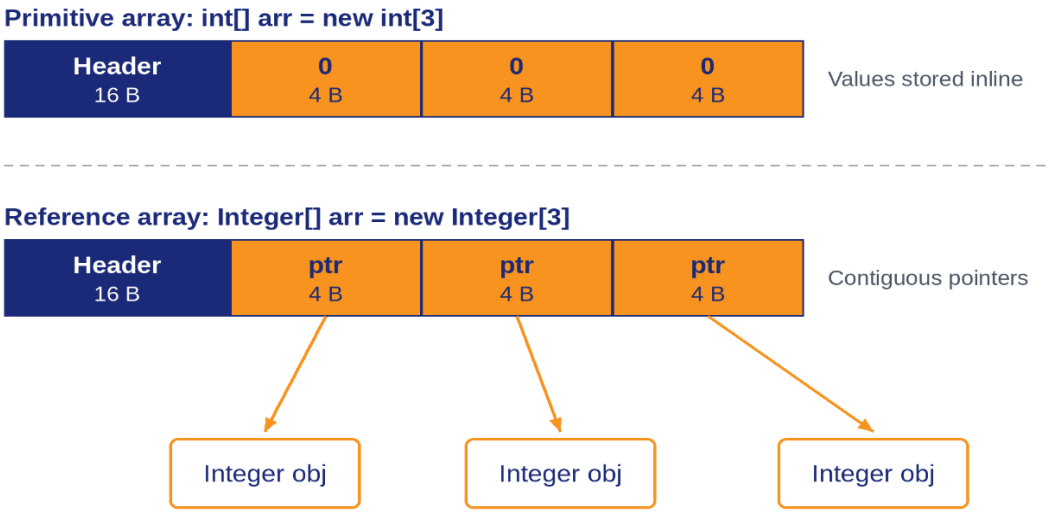
For int[]: Address(A[i]) = Base + 16 + (i × 4) bytes.
For long[] / double[]: Address(A[i]) = Base + 16 + (i × 8) bytes.

Multiplication and addition are single-cycle ALU operations, so indexing takes the same time whether the array holds ten elements or ten million.

PRO TIP

Worked example. A long[] starts at address 0x1000. Element a[10] is at 0x1000 + 16 + (10 × 8) = 0x1000 + 96 = **0x1060**.

1.3 Primitive vs. Reference Arrays



Objects scattered on the heap

- **Primitive arrays** (int[], char[], double[]): the values live inline inside the array's contiguous block.
- **Reference arrays** (Integer[], String[], Node[]): the array holds contiguous *references*; each one points to a separate object elsewhere on the heap. That means pointer chasing, fragmentation and cache misses.

1.4 Cache Lines, Spatial Locality & Memory Stride

Modern CPUs pull data from RAM into the L1/L2/L3 caches in chunks of **64 bytes** called **cache lines**. A 64-byte line holds up to 16 four-byte integers.

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]	a[11]	a[12]	a[13]	a[14]	a[15]

Figure 2: touching a[0] (orange) is a cache miss that loads the whole line; the next 15 reads are hits.

- **Sequential scan (stride 1):** the first access misses and loads the line; the next 15 accesses are L1 hits (about 1 ns each, versus about 100 ns from main RAM).
- **Object arrays / random lookups:** every dereference lands on an unrelated cache line, causing evictions and stalls.

KEY IDEA

Same Big-O, very different speed: looping over an int[] is usually several times faster than looping over an Integer[] of the same length, purely because of memory layout.

2. Declaration, Allocation, Initialization & I/O

2.1 Declaration vs. Instantiation

Declaration.java

```
int[] arr;          // Declaration: only a reference variable (4-8 B) on the stack.
                    // No array exists on the heap yet. arr == null.
arr = new int[5];   // Instantiation: contiguous block allocated on the heap.
                    // The JVM zero-initializes every element.
```

Default values on allocation

Type	Default value
byte, short, int, long	0
float, double	0.0
char	'\u0000' (the null character)
boolean	false
Reference types (Object, String, ...)	null

2.2 Creation & Traversal Patterns

Traversal.java

```
int[] a = new int[5];                 // five zeros
int[] b = {4, 8, 15, 16, 23};         // literal initializer
int n = b.length;                     // length is a field, not a method


for (int i = 0; i < n; i++) {         // index loop: use when you need i
    System.out.println(b[i]);
}
for (int x : b) {                     // for-each: read-only traversal
    System.out.println(x);
}
System.out.println(java.util.Arrays.toString(b));   // [4, 8, 15, 16, 23]
```

2.3 Fast I/O for Large Datasets

WATCH OUT

Standard java.util.Scanner relies on regular expressions and synchronized internal buffers. It is too slow once an array has $N \geq 10^5$ elements and leads to **Time Limit Exceeded**. Read raw bytes in big chunks instead.

FastScanner.java

```
import java.io.InputStream;

import java.io.IOException;

 public class FastScanner {
```

```
private final InputStream in;
private final byte[] buffer = new byte[1 << 16];     // 64 KB chunk buffer
private int head = 0;
private int tail = 0;
public FastScanner(InputStream in) {
    this.in = in;  }
private int readByte() throws IOException {
    if (head >= tail) {
        head = 0;
        tail = in.read(buffer, 0, buffer.length);
        if (tail <= 0) return -1;
    }
    return buffer[head++];
}
public int nextInt() throws IOException {
    int c = readByte();
    while (c <= ' ') {
        if (c == -1) return -1;
        c = readByte();  }
    boolean neg = (c == '-');
    if (neg) c = readByte();
    int res = 0;
    while (c >= '0' && c <= '9') {
        res = res * 10 + (c - '0');
        c = readByte(); }
    return neg ? -res : res; }}
```

Main.java (usage)

```
public static void main(String[] args) throws IOException {
    FastScanner fs = new FastScanner(System.in);
    int n = fs.nextInt();
    int[] a = new int[n];
    for (int i = 0; i < n; i++) a[i] = fs.nextInt();
    System.out.println(findMaximum(a)); }
```

PRO TIP

Do not call nextInt() more times than the input promises: at end of input it returns -1, which looks like a real value. For heavy output, collect lines in a StringBuilder and print once instead of calling System.out.println in a loop.

3. Core Array Algorithms: Patterns & Engineering Traps

3.1 Finding Extremes (Min / Max)

A classic mistake is to start the candidate at an arbitrary constant such as 0.

Wrong.java

```
// ANTI-PATTERN: fails if every element is negative
int max = 0;
for (int x : arr) if (x > max) max = x;
```

Robust version — O(N) time, O(1) space. Seed the answer with the first element and guard bad input.

FindMax.java

```
public static int findMaximum(int[] arr) {
    if (arr == null || arr.length == 0) {
        throw new IllegalArgumentException("Array cannot be null or empty.");
    }
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

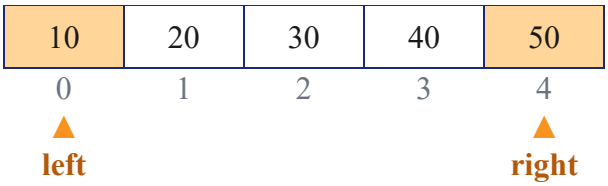
KEY IDEA

For the minimum, flip the comparison (arr[i] < min). Seeding with arr[0] works for any data; seeding with Integer.MIN_VALUE / MAX_VALUE also works but is easy to get wrong.

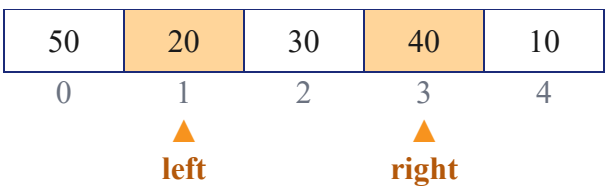
3.2 In-Place Two-Pointer Reverse

To reverse without allocating a second array (**O(1) auxiliary space**), place one pointer at each end, swap, and move both inward until they meet or cross.

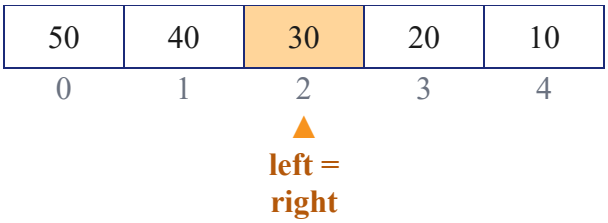
Initial: left = 0, right = 4 → swap(10, 50)



Step 1: left = 1, right = 3 → swap(20, 40)



Step 2: left == right = 2 → stop (the middle element stays put)



Reverse.java

```
public static void reverseInPlace(int[] arr) {
    if (arr == null || arr.length <= 1) return;
    int left = 0;
    int right = arr.length - 1;
    while (left < right) {
        // in-place swap through a temporary variable
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }}
```

Time is O(N) (about N/2 swaps); extra space is O(1).

3.3 All-Pairs Processing & Avoiding Symmetric Duplicates

To visit every unordered pair *{i, j}* exactly once, start the inner loop just after the outer index.

Pairs.java

```
// Distinct index pairs: {i, j} is the same pair as {j, i}
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        // process (arr[i], arr[j])
        // total pairs = n * (n - 1) / 2   ->   O(N^2)
    }}
```

PRO TIP
For n = 5 the pairs are (0,1) (0,2) (0,3) (0,4) (1,2) (1,3) (1,4) (2,3) (2,4) (3,4): that is 5 × 4 / 2 = **10**.
For n = 1000 it is 499,500 pairs, comfortably fast; for n = 10^5 it is about 5 × 10^9, far too slow.

4. Searching Paradigms: Linear vs. Binary Search

Feature	Linear Search	Binary Search
Precondition	None (works on unsorted data)	Strictly monotonic (sorted) data
Search-space reduction	Drops 1 element per check (N, N−1, N−2, ...)	Halves the remaining range (N, N/2, N/4, ...)
Time (worst)	O(N)	O(log N)
Time (best)	O(1) — first element	O(1) — exact midpoint match
Auxiliary space	O(1)	O(1) (iterative implementation)

4.1 Linear Search

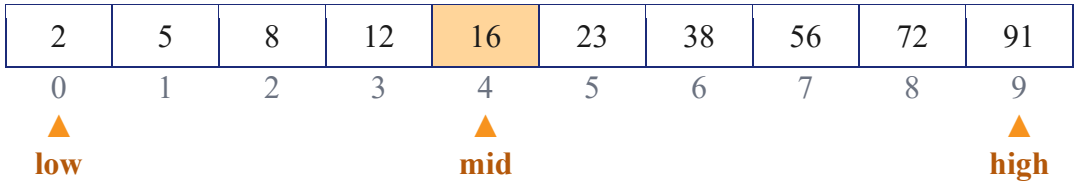
LinearSearch.java

```
public static int linearSearch(int[] arr, int target) {
    if (arr == null) return -1;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;          // first matching index
        }
    }
    return -1;                 // sentinel: target absent
}
```

4.2 Binary Search Deep Dive

Binary search finds a target in a **sorted** array by repeatedly halving the search window. Trace for **target = 23** in a 10-element array:

Step 1: low = 0, high = 9, mid = 4. arr[4] = 16 < 23 → target is in the right half, so low = mid + 1 = 5.



Step 2: low = 5, high = 9, mid = 7. arr[7] = 56 > 23 → target is in the left half, so high = mid − 1 = 6.

2	5	8	12	16	23	38	56	72	91
0	1	2	3	4	5	6	7	8	9
					▲ low		▲ mid		▲ high

Step 3: low = 5, high = 6, mid = 5. arr[5] = 23 == target → return index 5.

2	5	8	12	16	**23**	38	56	72	91
0	1	2	3	4	5	6	7	8	9
					▲ low = mid	▲ high			

The midpoint overflow bug

Midpoint.java

```
// BUGGY: if low + high exceeds 2,147,483,647 the sum wraps to a negative number
int mid = (low + high) / 2;
// SAFE: subtract first, so the intermediate value never overflows
int mid = low + ((high - low) >> 1);
```

Loop condition: low <= high vs. low < high

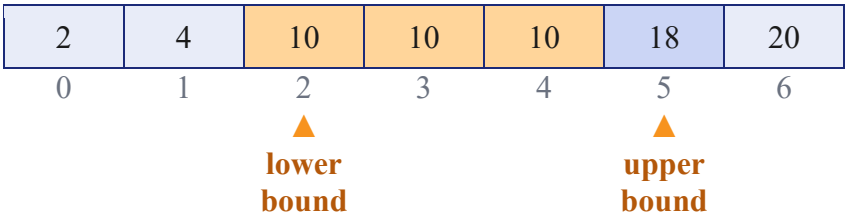
Style	Search interval	Loop / update rules	Typical use
low <= high	Closed [low, high] — every index in range can hold the answer	Update with mid + 1 and mid - 1. Loop ends when low = high + 1 (target absent).	Exact-match search
low < high	Half-open [low, high) — high is one past the end	Use high = mid (keep mid) and low = mid + 1.	Boundaries, insertion points, bounds

BinarySearch.java

```
public static int binarySearch(int[] arr, int target) {
    if (arr == null || arr.length == 0) return -1;
    int low = 0;
    int high = arr.length - 1;
    while (low <= high) {
        int mid = low + ((high - low) >> 1);
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;      // discard the left half
        } else {
            high = mid - 1;     // discard the right half
        }
    }
    return -1;                  // not found
}
```

4.3 Advanced Binary Search: Lower & Upper Bounds

With duplicate keys, a plain binary search may land on any copy. Interviews and contests usually ask for the first or last occurrence, which the two bound functions answer cleanly.



- **Lower bound:** first index where arr[i] >= target → index 2.
- **Upper bound:** first index where arr[i] > target → index 5.
- **Count of the target** = upperBound − lowerBound = 5 − 2 = 3.

BoundSearch.java

```
public class BoundSearch {
   // First index i with arr[i] >= target (arr.length if none exists)
   public static int lowerBound(int[] arr, int target) {
      int low = 0, high = arr.length;
      while (low < high) {
         int mid = low + ((high - low) >> 1);
         if (arr[mid] >= target) {
            high = mid;          // answer may be mid itself: keep it
         } else {
            low = mid + 1;
         } }
      return low; }
   // First index i with arr[i] > target (arr.length if none exists)
   public static int upperBound(int[] arr, int target) {
      int low = 0, high = arr.length;
      while (low < high) {
         int mid = low + ((high - low) >> 1);
         if (arr[mid] > target) {
            high = mid;
         } else {
            low = mid + 1;
}}
      return low; }}
```

KEY IDEA

Recipes. First occurrence: `i = lowerBound(a, t)`, valid only if `i < a.length && a[i] == t`. Last occurrence: `upperBound(a, t) - 1` (check the same way). Insertion position: `lowerBound(a, t)`.

5. Basic Complexity Intuition

5.1 Asymptotic Growth Orders

Growth order

O(1) < O(log N) < O(sqrt N) < O(N) < O(N log N) < O(N^2) < O(2^N) < O(N!)
-->
slower as N grows

Complexity.java

```
// O(1): constant, same work for any input size
int val = arr[0];
// O(log N): problem size shrinks by a constant factor each step
while (n > 1) { n /= 2; }
// O(N): one pass over the input
for (int i = 0; i < n; i++) { }
// O(N^2): nested pass over the input
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { }
}
```

What the classes mean for N = 10^6 (roughly):

Complexity	Operations	Verdict in 1 second (about 10^8 ops)
O(1)	1	Instant
O(log N)	≈ 20	Instant
O(N)	10^6	Easy
O(N log N)	≈ 2×10^7	Fine
O(N^2)	10^{12}	Far too slow

5.2 The 1-Second Rule for Online Judges

Most platforms (LeetCode, Codeforces, HackerRank) allow about **1 second**, which is roughly **10^8 basic operations** in Java. Read the constraints first and let them choose your algorithm:

Maximum N in constraints	Expected complexity	Typical technique
N ≤ 10 to 12	O(N!) or O($2^N \cdot N$)	Brute-force permutations, bitmask DP

$N \leq 20$ to 25	$O(2^N)$	Backtracking, subset generation
$N \leq 500$ to 10^3	$O(N^2)$	Dynamic programming, nested loops, matrices
$N \leq 10^5$ to 10^6	O(N log N) or O(N)	Binary search, sorting, two pointers, sliding window
$N \geq 10^9$	O(log N) or O(1)	Math formulas, binary search on answer, matrix exponentiation

PRO TIP

Example. "$N \leq 2 \times 10^5$, find two elements that sum to K." An all-pairs loop is about 2×10^{10} operations: too slow. Sort once (O(N log N)) and use two pointers or binary search instead.

6. Common Mistakes & Debugging Checklist

Mistake	What goes wrong	Fix
int max = 0;	Wrong answer when all values are negative	Seed with arr[0] after an empty check
(low + high) / 2	Integer overflow on huge ranges	low + ((high - low) >> 1)
Mixing low <= high with high = mid	Infinite loop when low == high	Pair <= with mid ± 1; pair < with high = mid
Binary search on unsorted data	Silently returns wrong results	Sort first, or use linear search
i <= arr.length	ArrayIndexOutOfBoundsException (off by one)	Use i < arr.length
int[] b = a;	No copy is made: both names share one array	Arrays.copyOf(a, a.length) or a.clone()
Inner pair loop starts at 0	Counts (i, i) and both (i, j) and (j, i)	Start with j = i + 1
Summing ints into an int	Overflow beyond about 2.1 × 109	Accumulate in a long
Comparing Integer with ==	Compares references; only correct by luck for -128..127	Use .equals() or unbox first
Scanner for $N \geq 10^5$	Time Limit Exceeded	FastScanner or BufferedReader
Missing null / empty guard	NullPointerException or arr[0] crash	Validate input before touching elements
Allocating a new array to "reverse in place"	O(N) extra space, may violate the problem	Two-pointer swap

7. Practice Questions

Warm-up (concepts and tracing)

Q1. Given int[] a = new int[4]; boolean[] b = new boolean[2]; String[] s = new String[2]; what do a[2], b[0] and s[1] contain?

Q2. A long[] starts at address 0x1000 and the header is 16 bytes. What is the address of a[10]?

Q3. Trace binary search for target 8 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]. Write down low, high and mid at every step.

Q4. Give the time complexity of (a) `for (int i = 1; i < n; i *= 2)` and (b) `for i in 0..n-1, for j in 0..i-1`.

Core (implement it)

Q5. Find the **second largest distinct** value in one pass. Decide what to return for an array with fewer than two distinct values.

Q6. Find the minimum and maximum in a single pass. *Challenge:* use fewer than 2N comparisons by comparing elements in pairs first.

Q7. Reverse the sub-array `arr[l..r]` in place. Then rotate the whole array right by *k* using three reversals (whole array, first k, remaining N−k).

Q8. Count pairs `i < j` with `a[i] + a[j] == K` in $O(N^2)$. How many pairs does the loop examine for N = 1000? Then solve it for a **sorted** array in O(N) with two pointers.

Q9. Using `lowerBound` and `upperBound`, return the first index, last index and count of a target in a sorted array with duplicates (e.g. [2,4,10,10,10,18,20], target 10 → 2, 4, 3; absent target → -1, -1, 0).

Q10. Search-insert position: return the index at which a target is found, or where it would be inserted to keep the array sorted.

Challenge (think about constraints)

Q11. $N \le 2 \times 10^5$. Decide whether $O(N^2)$ or O(N log N) will pass, then design an approach for "does any pair sum to K?".

Q12. Compute floor(sqrt(x)) for x up to $2^{31} - 1$ using binary search on the answer, without `Math.sqrt`. Why must `mid * mid` be computed in a `long`?

Q13. Find the minimum element of a sorted array that has been rotated at an unknown pivot (distinct values) in O(log N).

Q14. Read N and then N integers with `FastScanner` and print the maximum, the minimum, and how many times the maximum occurs.

ANSWER KEY (WARM-UPS)

- **Q1:** 0, false, null.
- **Q2:** 0x1000 + 16 + 10 × 8 = 0x1060.
- **Q3:** mid = 4 (16 > 8, high = 3); mid = 1 (5 < 8, low = 2); mid = 2 (8 found). Index 2 after 3 probes.
- **Q4:** (a) O(log N); (b) the inner loop runs 0 + 1 + ... + (n−1) = n(n−1)/2 times, so $O(N^2)$.
- **Q8 (count):** 1000 × 999 / 2 = 499,500 pairs.

8. Summary

- A Java array is a **heap object**: a 16-byte header, then contiguous elements, padded to 8 bytes.
- Indexing is **O(1)** because address = base + 16 + i × element size.

- Primitive arrays store values inline (cache-friendly); reference arrays store pointers (cache-hostile).
- Use FastScanner for $N \geq 10^5$; seed max/min with arr[0]; guard null and empty input.
- Reverse with two converging pointers; visit pairs with $j = i + 1$.
- Binary search needs **sorted** data, a safe midpoint and a consistent loop condition; bounds handle duplicates.
- Let the constraints pick the complexity: about 10^8 simple operations per second.

Operation	Time	Extra space
Access / update arr[i]	O(1)	O(1)
Find max / min	O(N)	O(1)
Reverse in place	O(N)	O(1)
Linear search	O(N)	O(1)
Binary search, lower bound, upper bound	O(log N)	O(1)
All pairs	$O(N^2)$	O(1)

9. Curated Reference Study Links

Memory & bytecode foundations

- Baeldung — Memory Layout of Java Arrays and Objects

Searching algorithms & two pointers

- Take U Forward (Striver) — Binary Search Master Playlist (step-by-step lower/upper bounds and binary search on answer ranges).

Algorithmic complexity

- Big-O Cheat Sheet — Time & Space Complexity Reference Guide

CONCLUSION

Arrays are fundamental data structures. Understanding their memory layout, searching techniques, in-place operations, and time complexity helps you write efficient and reliable solutions. The key is to choose the right approach based on the problem and its constraints.